# NOTED!

# Dokumentáció

# 1.Bevezetés

## 1.1 A Projekt Célja

A Noted! egy böngészőalapú projektmenedzsment alkalmazás, amely egyetlen, bejelentkezéshez kötött munkaterületen belül ötvözi a feladatkövetést (Kanban tábla) és a folyamatok vizuális ábrázolását (folyamatábra-szerkesztő), valamint a jegyzetek és képek tárolását (note és picture modul) az egyszerűbb projekt tervezéshez. A cél az, hogy a felhasználó egy helyen tudjon tervezni, vizualizálni és nyomon követni munkafolyamatokat anélkül, hogy több különálló eszköz vagy oldal között kelljen váltogatnia.

## 1.2 A Megoldandó Probléma

A személyes projekteket tervező felhasználók gyakran több, egymástól független eszközt használnak: egy feladatlistát az egyik alkalmazásban, egy kézzel rajzolt vagy külön tárolt folyamatábrát egy másikban (vagy másik oldalon), és nincs egyetlen közös hely, ahol mindkettő ugyanahhoz a projekthez kapcsolódna. A NOTED! ezt úgy oldja meg, hogy minden projekthez saját munkaterületet rendel, amely egyszerre tartalmazza a Kanban és Folyamatábra, Kép és Jegyzet modulokat, egy közös felhasználói fiók és adatbázis mögött.

## 1.3 Célcsoport / felhasználók

Az alkalmazás elsődleges célcsoportja a diákok és egyéni tervezők, akiknek könnyen kezelhető vizuális projekttervezésre van szükségük, a vállalati szintű projektmenedzsment eszközök túlzott komplexitása és költsége nélkül. A rendszer jelenleg egyetlen felhasználói szerepkört támogat: a bejelentkezett fiók

tulajdonosát, aki kizárólag a saját maga által létrehozott projektekhez fér hozzá.

## 1.4 A rendszer fő funkciói

**Regisztráció, bejelentkezés és kijelentkezés:** munkamenet alapú session cookie hitelesítéssel, bcrypt jelszó hasheléssel.
**Projektek kezelése:** létrehozás, átnevezés, és megnyitás, minden projekt önálló, elkülönített munkaterület.
**Kanban modul:** táblák négy fix oszloppal (To Do, Doing, Done, Delayed), drag and drop kártyákkal, amelyekhez cím, leírás és opcionális kép is tartozhat.
**Folyamatábra modul:** átméretezhető rajzvásznak, amelyeken egy folyamatábrához szükséges alakzatok helyezhetők el és köthetők össze nyilakkal, amelyek a mozgatás közben is helyükön maradnak.
**Note modul:** Kisebb jegyzettömbb, aminek a törzsébe és címsorába lehet írni.
**Picture modul:** Kép URL beillesztésével képet megjeleníteni, és annak címet adni.
**Drag and Drop pozícionálás és átméretezés:** táblákhoz, folyamatábrákhoz és alakzatokhoz egyaránt, adatbázisban perzisztálva.
**Felhasználónkénti és projektenkénti adatelkülönítés:** minden API kérésnél szerveroldalon ellenőrzött jogosultságokkal, nem csak kliensre bízva.

## 1.5 Egyéb Tudnivalók

A Projekt fejlesztése folyamán a mesterséges intelligencia kizárólag szakirodalom kutatásra és példaprogramok létrehozására lett használva, amelyek a projekt fejlesztése során nem kerültek bele a programba, a kész terméket nem befolyásolták.

A programban felhasznált képek Google-ről lettek letöltve.

# 2.Követelmények

## 2.1 Funkcionális követelmények

**Regisztráció:** látogató fiókot hozhat létre felhasználónévvel, email címmel és jelszóval.
**Bejelentkezés / Kijelentkezés:** regisztrált felhasználó bejelentkezhet és kijelentkezhet.
**Projektek kezelése:** a bejelentkezett felhasználó létrehozhat, átnevezhet, törölhet, megnyithat kizárólag saját projekteket.
**Kanban táblák:** a munkaterületen belül létrehozhatók, mozgathatók, átméretezhetők és törölhetők.
**Kártyák kezelése:** kártyák hozzáadhatók egy táblához, oszlopok között mozgathatók (drag and drop), és törölhetők.
**Folyamatábrák:** létrehozhatók, mozgathatók, átméretezhetők és törölhetők.
**Jegyzetek:** létrehozhatók, mozgathatók, átnevezhetők, szerkeszthetők.
**Képek:** létrhozhazók, mozgathatók, átnevezhetők, szerkeszthetők, kép feltöltése kép-URL segítségével.
**Alakzatok kezelése:** egy folyamatábrán alakzatok hozzáadhatók, mozgathatók törölhetők, és átnevezhetők, valamint nyíllal összeköthetők amelyek mozgás közben is helyükön maradnak.
**Adatperzisztencia:** a fenti összes módosítás MySQL adatbázisban tárolódik, és oldal-újratöltés után is visszaáll
Jogosultságkezelés: egy felhasználó kizárólag a saját projektjeihez tartozó adatokat olvashatja vagy módosíthatja.

## 2.2 Nem funkcionális követelmények

**Biztonság:** a jelszavak soha nem kerülnek nyílt szövegként tárolásra (bcrypt hashelés); minden adatmódosító kérés hitelesítve van, és a tulajdonos felhasználó jogosultság ellenőrizve van.
**Adatperzisztencia:** minden felhasználó által létrehozott tartalom túléli a szerver újraindítását és az oldal újratöltését, a MySQL adatbázisnak köszönhetően.
**Használhatóság:** közvetlen drag and drop alapú interakciók a táblák, kártyák, folyamatábrák és alakzatok kezelésére.
**Karbantarthatóság:** a kliensoldali kód funkcionális modulokra van bontva (hitelesítés, projektlista, Kanban, Folyamatábra), amelyek egy közös API alapcímet és projekt-kontextust osztanak meg egymással.
**Info gomb a munkaterületen:** a workspace.html-en jobb fenn található (?)-gomb a modulok irányításához / kezeléséhez tartozó információkat jeleníti meg a felhasználónak, mind telefonon mind számítógépen.

## 2.3 Felhasználói szerepkörök

A jelenlegi verzió egyetlen felhasználói szerepkört valósít meg: a bejelentkezett fiók tulajdonosát. Nincs külön adminisztrátori szerepkör, minden hitelesített felhasználó azonos képességekkel rendelkezik, a hozzáférése pedig kizárólag a saját maga által létrehozott projektekre korlátozódik. Ez a korlátozás nem csak a felhasználói felületen érvényesül, hanem minden API-végponton szerveroldalon is ellenőrzésre kerül, így egy felhasználó még a kérések közvetlen módosításával sem férhetne hozzá más projektjéhez.

# 3. Rendszerterv

## 3.1 Architektúra

A NOTED! klasszikus háromrétegű webarchitektúrát követ:
**Kliensoldal:** statikus HTML/CSS/JavaScript fájlok, amelyeket a böngésző jelenít meg, nincs build folyamat vagy frontend keretrendszer.
**Alkalmazásréteg:** Node.js / Express alapú REST API, amely a hitelesítést, jogosultságkezelést valósítja meg.
**Adatréteg:** MySQL relációs adatbázis, amelyhez a szerver egy connection poolon keresztül fér hozzá.

A kliens soha nem kommunikál közvetlenül adatbázissal, minden olvasási vagy írási művelet a REST API-n keresztül történik, amely minden adat elérése előtt ellenőrzi a jogosultságot.

## 3.2 A rendszer komponensei

| Komponens | Fájlok | Feladat |
|---|---|---|
| Nyitóoldal | index.html / index.css | Nyilvános bemutatkozó oldal, bejelentkezés / regisztráció |
| Regisztráció | register.html / register.css / register.js | Fiók létrehozása, jelszó megerősítés ellenőrzése |
| Bejelentkezés | signin.html / signin.css / singin.js | Bejelentkezési űrlap |
| Főoldal (homepage) | homepage.html / homepage.css / homepage.js | A bejelentkezett felhasználó projektjeinek listázása, |

| | | |
|---|---|---|
| | | projekt létrehozása / átnevezése / törlése / megnyitása, felhasználónév változtatása és kijelentkezés |
| Munkaterület (workspace) | workspace.html / workspace.css / workspace.js | Egy adott projekt közös vászna, itt jelennek meg együtt a projekttervező modulok |
| Kanban Modul | kanbanModule.js / kanbanModule.css | Táblák, oszlopok, kártyák, drag and drop működés, kártya részletek ablak |
| Folyamatábra Modul | flowchartModule.js / flowchartModule.css | Rajzvásznak, alakzatok, SVG összekötések, mozgatás / átméretezés |
| Jegyzet modul | noteModule.js / noteModule.css | Jegyzetírás, címsor átírása |
| Kép modul | pictureModule.js / pictureModule.css | Képek beszúrása, modul címének átírása |

| | | |
|---|---|---|
| Szerver | server.js | Express alkalmazás: middleware-ek, REST endpointok, munkamenetkezelés, adatbázis-pool |

## 3.3 Kommunikáció a komponensek között

A böngésző és a szerver kizárólag HTTP-n keresztül, a fetch API segítségével, JSON formátumú adatcsomagokkal kommunikál. A hitelesítési állapotot egy munkamenet-cookie (connect.sid) hordozza automatikusan minden azonos eredetű kérésnél, miután a felhasználó bejelentkezett, a kliensoldali kódnak nem kell tokent kezelnie vagy tárolnia. A szerver a MySQL adatbázissal egy mysql2 connection-poolon keresztül kommunikál, és minden olyan lekérdezésnél, amely felhasználó által megadott adatot tartalmaz, paraméterezett lekérdezéseket használ, ami megvédi a rendszert az SQL-injection-től.

## 3.4 REST API

Minden végpont a /api útvonal alatt érhető el. Az „Auth” jelöléssel ellátott végpontok érvényes bejelentkezett munkamenetet igényelnek, enélkül a kérés HTTP 401 választ kap.

| Metódus | Végpont | Funkció |
|---|---|---|
| POST | /api/register | Fiók létrehozása, jelszó hashelése, alapértelmezett projekt létrehozása, |

| | | |
|---|---|---|
| | | munkamenet indítása |
| POST | /api/login | Hitelesítő adatok ellenőrzése, munkamenet indítása |
| POST | /api/logout | Munkamenet megszüntetése |
| GET | /api/me (Auth) | A bejelentkezett felhasználó profiladatainak lekérdezése |
| PUT | /api/me (Auth) | A bejelentkezett felhasználó nevének módosítása |
| GET | /api/projects (Auth) | A felhasználó projektjeinek listázása |
| POST | /api/projects (Auth) | Új projekt létrehozása |
| PUT | /api/projects/:id (Auth) | Egy saját projekt átnevezése |
| GET | /api/boards?projectId = (Auth) | Egy projekt Kanban tábláinak listázása |
| POST | /api/boards (Auth) | Tábla létrehozása egy saját projektben |
| PUT | /api/boards/:id (Auth) | Tábla címének, tartalmának és pozíciójának frissítése |
| DELETE | /api/boards/:id (Auth) | Tábla törlése |

| | | |
|---|---|---|
| GET | /api/charts?projectId= (Auth) | Egy projekt folyamatábráinak listázása |
| POST | /api/charts (Auth) | Folyamatábra létrehozása egy saját projektben |
| PUT | /api/charts/:id (Auth) | Folyamatábra tartalmának és pozíciójának frissítése |
| DELETE | /api/charts/:id (Auth) | Folyamatábra törlése |
| GET | /api/health | Adatbázis kapcsolat ellenőrzése |
| GET | [/api/notes]?projectId= és [/api/pictures]?projectId= (Auth) | A projekt jegyzeteinek / képeinek listázása |
| POST | [/api/notes] és [/api/pictures] (Auth) | Jegyzet / kép létrehozása egy saját projektben |
| PUT | [/api/notes]/:moduleId és [/api/pictures]/:moduleId (Auth) | Jegyzet / kép címének, tartalmának és pozíciójának módosítása |
| DELETE | [/api/notes]/:moduleId és [/api/pictures]/: moduleId (Auth) | Jegyzet / kép törlése |

## 3.5 **Alkalmazott technológiák**

| Réteg | Technológia |
|---|---|

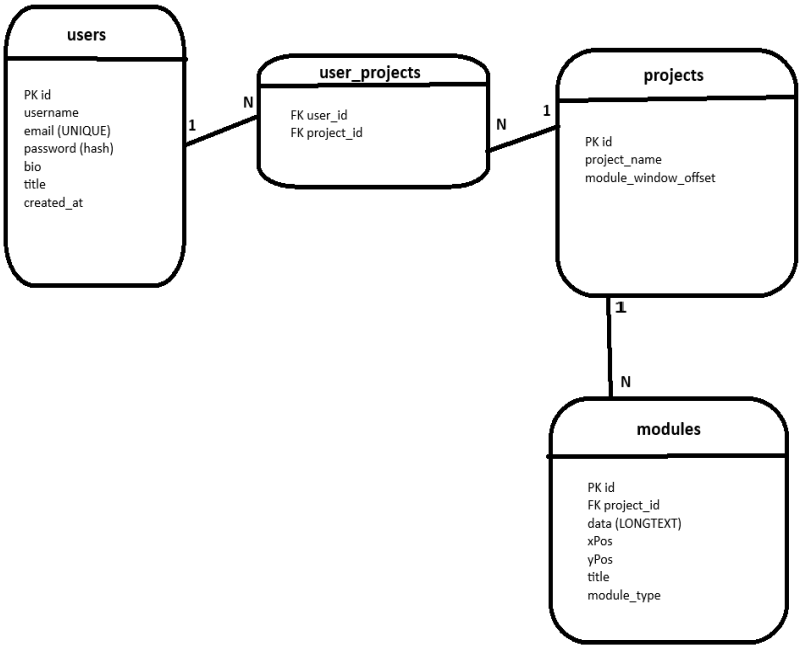
| | |
|---|---|
| Futtatókörnyezet | Node.js |
| Webes keretrendszer | Express |
| Adatbázis meghajtó | mysql2 (promise API, connection pool) |
| Adatbázis | MySQL / MariaDB (fejlesztés XAMPP alatt) |
| Hitelesítés | express-session (cookie alapú munkamenet), bcrypt (jelszó hashelés) |
| Egyéb middleware | cors, dotenv |
| Kliensoldal | Tiszta HTML5, CSS3, vanilla JavaScript, keretrendszer és build folyamat nélkül |
| Kliens-szerver protokoll | REST HTTP felett, JSON adatcsomagokkal |

# 4. Adatbázis

## 4.1 Adatmodell

Az adatbázis négy táblából áll. Egy felhasználó több projektet is birtokolhat, egy projektnek pedig több felhasználója is lehet, ezt egy több a többhöz kapcsolat modellezi a user_projects kapcsolótáblán keresztül. Minden projekthez tetszőleges számú modul tartozhat: egyetlen közös modules tábla tárolja mind a Kanban táblákat, Folyamatábrákat, Jegyzeteket és Képeket amelyeket a module_type oszlop különböztet meg egymástól, mivel mind a négy modultípus azonos tárolási szerkezetet használ: pozíció (xPos, yPos), cím, valamint egy data mező, amely a szerializált HTML tartalmat tárolja.

## 4.2 ER-diagram



A fenti képen látható táblák közötti kapcsolat: a users és projects táblák a user_projects kapcsolótáblán keresztül állnak több a többhöz viszonyban, míg a projects és modules táblák közt egy közvetlen egy a többhöz kapcsolat van a project_id idegen kulcson keresztül.

## 4.3 Táblák részletes leírása

**users**

| Oszlop | Típus | Kulcs | Leírás |
|---|---|---|---|
| id | INT | PK | Autómatikusan kiadott felhasználó azonosító |
| username | VARCHAR(50) | | Megjelenített név (főoldalon szerkeszthető) |
| email | VARCHAR(100) | UNIQUE, NOT NULL | Bejelentkezési azonosító |
| password | VARCHAR(255) | NOT NULL | bcrypt hash, soha nem a nyers jelszó |
| bio | VARCHAR(255) | | Jövőbeli profil leírásnak fenntartva |
| title | VARCHAR(50) | | Jövőbeli profil beosztásnak fenntartva |
| created_at | TIMESTAMP | DEFAULT NOW | Fiók létrehozásának időpontja |

**projects**

| Oszlop | Típus | Kulcs | Leírás |
|---|---|---|---|
| id | INT | PK | Automatikusan kiadott projekt azonosító |
| project_name | VARCHAR(50) | NOT NULL | A főoldalon megjelenő projektnév |
| module_window_offset | BIGINT | DEFAULT 0 | Jövőbeli vászon eltolásnak fenntartva |

**modules**

| Oszlop | Típus | Kulcs | Leírás |
|---|---|---|---|
| id | INT | PK | Automatikusan kiadott modul azonosító |
| project_id | INT | FK -> projects.id | A modult birtokló projekt |
| data | LONGTEXT | | A tábla vagy folyamatábra szerializált belső HTML tartalma |
| xPos / yPos | BIGINT | Default 0 | A modul pozíciója a közös vásznon |
| title | TINYTEXT | | A tábla vagy folyamatábra címe |
| module_type | VARCHAR(20) | DEFAULT 'kanban' | Megkülönbözteti a Kanban táblákat, Folyamatábrákat, Jegyzeteket és Képeket |

**user_projects**

| Oszlop | Típus | Kulcs | Leírás |
|---|---|---|---|
| user_id | INT | FK -> users.id | A projekteket birtokló felhasználó |
| project_id | INT | FK -> projects.id | A birtokolt projekt |

## 4.4 Kapcsolatok

**users <-> projects:** több a többhöz, a user_projects táblán keresztül megvalósítva.

**projects -> modules:** egy a többhöz (a modules.project_id a projects.id-ra mutat).

-Minden tábla, folyamatábra és projekt végpont ezeken a kapcsolatokon keresztül vezeti le a jogosultságot minden egyes kérésnél, nem pedig a kliens által küldött azonosítóban bízva, ez teszi lehetővé a felhasználónkénti adatelkülönítés tényleges, szerveroldali kikényszerítést.

## 4.5 Adatbázis létrehozása / dump

```sql
CREATE DATABASE NOTED;


USE NOTED;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50),
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    bio VARCHAR(255),
    title VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE projects (
    id INT AUTO_INCREMENT PRIMARY KEY,
    project_name VARCHAR(50) NOT NULL,
    module_window_offset BIGINT DEFAULT 0
);
```

```
CREATE TABLE modules (
    id INT AUTO_INCREMENT NOT NULL PRIMARY KEY,
    project_id INT NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(id),
    data LONGTEXT,
    xPos BIGINT DEFAULT 0,
    yPos BIGINT DEFAULT 0,
    title TINYTEXT,
    module_type VARCHAR(20) NOT NULL DEFAULT 'kanban'
);

CREATE TABLE user_projects (
    user_id INT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id),
    project_id INT NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(id)
);
```

# 5. Felhasználói felület

## 5.1 Képernyők

A projektben több oldalt is látogathat a felhasználó:

| Oldal | Tartalma |
|---|---|
| index.html | A program landing page-e, a felhasználó itt olvashat a program tartalmáról, és itt tud átmenni a regisztrációs / bejelentkező felületre / oldalra |
| signin.html / register.html | A felhasználó itt tud regisztrálni / bejelentkezni a felületre |
| homepage.html | A felhasználó erről az oldalról tudja megnyitni a létező projektjeit, valamint újakat tud csinálni, vagy törölni, és itt tudja a felhasználói nevét és képét megváltoztatni |
| workspace.html | A felhasználó itt tudja a projektjeit megtervezni, itt találhatóak a projekt tervező modulok |

## 5.2 Navigáció

Az alkalmazás oldalai egy lineáris folyamatot alkotnak, amelyben a felhasználó a nyitóoldalról a hitelesítésen és a projektlistán keresztül jut el a munkaterületig.

| Honnan | Művelet | Hova |
|---|---|---|

| | | |
|---|---|---|
| Nyitóoldal (index.html) | "Sign in" gomb | signin.html |
| Nyitóoldal (index.html) | "Register" gomb | register.html |
| signin.html / register.html | Link a másik oldalra ("Register" / "Sign in") | A másik hitelesítési oldal |
| signin.html | Sikeres bejelentkezés | homepage.html |
| register.html | Sikeres regisztráció (a felhasználó azonnal be is van léptetve) | homepage.html |
| homepage.html | Kattintás egy projektkártyára | workspace.html?id=<projekt azonosító><br>(A kijelölt projekt munkaterületére) |
| homepage.html | "Log out" a profil legördülőmenüjében | signin.html |
| workspace.html | "Noted" gombra kattintva | homepage.html |

**A navigációt két védelmi szabály tarja rendben:**

**Hitelesítési őr (AuthGuard):** a homepage.html és a workspace.html betöltéskor meghívja a GET /api/me végpontot, és érvénytelen munkamenet esetén a signin.html oldalra irányít. Így a védett oldalak kijelentkezés után sem érhetők el a böngésző "vissza" gombjával vagy közvetlen URL-beírással.

**Projekt azonosító ellenőrzése:** ha a workspace.html-t ?id= paraméter nélkül nyitják meg, a rendszer visszairányít a homepage.html oldalra, mert nincs megjeleníthető projekt. Ha a paraméterben szereplő

projekt nem a bejelentkezett felhasználóé, a szerver 403-as választ ad, és az oldalon nem jelenik meg adat.

A projekt azonosítója URL-ben utazik, ezért egy adott projekt munkaterülete könyvjelzőzhető és megosztható saját fiókon belül.

## 5.3 Reszponzív működés

A Noted felülete mobiltelefontól asztali monitorig használható. A reszponzív működés kizárólag CSS-sel valósul meg, négy eszköz kombinációjával: rugalmas elrendezések (Flexbox, CSS Grid), folyékony méretezés (min(), clamp(), vw, dvh), media query-k és érintésbarát finomhangolás.

**Desktop first megközelítés.** Az alap stílus az asztali nézethez készült, a kisebb kijelzőket max-width media query-k kezelik. Például a Note modul alapból min(320px, 34vw) széles, 760px alatt pedig min(240px, calc(100vw – 32px)).

Folyékony méretezés fix értékek helyett. Ahol lehetett, a méretek a képernyőhöz igazodnak.

**Az elrendezés átalakul, nem csak összemegy:**

A munkaterület oldalsávja asztali gépen függőleges, a bal oldalon rögzített sáv. 760 px alatt vízszintes, tördelhető sávvá válik a képernyő tetején, a tartalom (#sheet) bal margója pedig nullára áll.

A Kanban tábla oszlopai asztali gépen egymás mellett vannak (flex). 1024px alatt kétoszlopos grid lesz belőlük, 420px alatt egyoszlopos.

A projektkártyák (.project-box) 600 px alatt soronként kettő, 400px alatt soronként egy darab látszik.

A flowchart alakzatválasztó menüje repeat(auto-fit, minmax(92px, 1fr)) rácsot használ, így az ikonok száma magától igazodik a menü szélességéhez.

Kisebb képernyőn kisebb elemek. A flowchart alakzatai például 100x60 px-ről 76x46-ra majd 64x40-re kicsinyednek. A modulok fejlécei, gombjai belső margói és betűméretei szintén csökkennek.

Elemek elrejtése. A főoldal dekoratív elemei (lebegő jegyzetkártyák, ceruzák, fogaskerék) 600px alatt eltűnnek, ahogy a funkciókártyák illusztrációi is. Ez a hely és olvashatóság miatt kell.

**Érintőképernyő támogatás:**

A húzható fejléceken touch-action szerepel, hogy a húzás érintéssel is működjön, és ne görgesse közben az oldalt.

Az átméretező fogantyú mobilon nagyobb (16px -> 20px)

Az érintőképernyős gombok minimális mérete 30-40px

Ahol a megjelenés hoverre épül, ott érintőeszközön másképp működik. A "Létrehozás" gomb szövege mobilon folyamatosan látható, mert nincs hover.

Modern viewport-egységek és biztonságos területek. A teljes magasságú elemek 100dvh-t használnak, a 100vh mellett tartalékként. A dvh figyelembe veszi, hogy a mobilböngésző címsora megjelenik és eltűnik, így az oldal nem lóg ki az aljáról. Az env(safe-area-inset-*) értékek megakadályozzák, hogy tartalmat kivágja a telefon lekerekített sarka vagy bevágása.

Töréspontok oldalanként:

| Oldal | Töréspontok | Legfontosabb változások |
|---|---|---|
| Nyitóoldal | 900 / 600 / 380px | A hero egyoszlopos lesz, a navigáció tördelődik, a vélemények 3-2-1 oszlopra váétanak, a lábléc 3-2-1 oszlopra |

| | | |
|---|---|---|
| Regisztráció, belépés | 600px | A kártya két panelje egymás alá kerül (flex-direction: column) |
| Projektek listája | 760 / 600 / 400px | Kisebb margók és méretek, kártyák 2-1 oszlopban, a törlési dialógus mobilra méretezve |
| Munkaterület | 760 / 420px | Az oldalsáv vízszintes sávvá alakul, a tart |
| Kanban, flowchart | 1024 / 760 / 420px | Kisebb modulok, oszlopok és alakzatok, a modális ablakok hörgethetők |
| Note és Picture modul | 760px | Szűkebb modul, kisebb fejléc és gombok, dinamikus magasság (clamp, dvh) |

A nyitóoldal a body { zoom: 1.2 } tulajdonságot használja, ami böngészőktől függően kissé eltérően skálázhat.

## 5.4 Felhasználói szerepkörök

Egyetlen felhasználói szerepkör létezik, ami pedig a bejelentkezett felhasználó, a nem bejelentkezett felhasználó (guest) az csak a nyitó oldalt látja, ahol átirányítva lesz a bejelentkezési illetve regisztrációs felületre.

# 6.Backend

## 6.1 Szerver felépítése

A server.js egyetlen Express alkalmazásként valósítja meg a backendet. A middleware-ek az alábbi sorrendben kerülnek regisztrálásra: CORS, JSON törzs feldolgozás (body parsing), munkamenet kezelés (express-session), majd a statikus fájlok kiszolgálása a public / könyvtárból. Egy mysql2 conection-pool egyszer jön létre a szerver indulásakor, és minden végpont ezt használja újra, nem nyit külön kapcsolatot minden egyes kéréshez. A végpontok erőforrás szerint vannak csoportosítva (hitelesítés, projektek, táblák, folyamatábrák, health check, valamint a jegyzet és kép modul egyetlen közös függvényen osztoznak), ezt követi egy záró hibakezelő middleware, majd az app.listen hívás, amely elindítja a szervert.

## 6.2 API végpontok

A teljes végpont táblázat a 3.4. fejezetben található. Összefoglalva: a hitelesítési végpontok (/api/register, /api/login, /api/logout, /api/me) nem igényelnek projekt kontextust, míg a projekt, a tábla, folyamatábra, jegyzet, és kép végpontok mindegyike a bejelentkezett felhasználó munkamenetéhez (session) kötötten működik, és minden írási / olvasási művelet előtt ellenőrzi a jogosultságot.

## 6.3 Hitelesítés / jogosultságkezelés

**Jelszó tárolás:** a jelszavak bcrypt algoritmussal kerülnek hashelésre, mielőtt bármi is bekerülne az adatbázisba, nyers jelszó soha nem kerül tárolásra vagy naplózásra.

**Bejelentkezés:** a megadott jelszót a bcrypt.compare függvény veti össze a tárolt hash-el, sikeres egyezés esetén a felhasználó azonosítója bekerül a munkamenetbe (req.session.userId).

**requireAuth middleware:** minden olyan végponton alkalmazva van, amely felhasználói adatot olvas vagy módosít, érvényes munkamenet hiányában HTTP 401 választ ad vissza, mielőtt a kérés egyáltalán elérné az adatbázis lekérdezést.

**Tulajdonjog ellenőrzés minden kérésnél:** minden tábla, folyamatábra és projekt végpont a user_projects táblán keresztüli JOIN-nal újra leellenőrzi a jogosultságot, a kliens által küldött projectId vagy erőforrás azonosító kizárólag egy olyan WHERE feltételben kerül felhasználásra, amely a munkamenet felhasználó azonosítóját is ellenőrzi. Ennek köszönhetően egy felhasználó még az azonosítók közvetlen kitalálásával/módosításával sem férhet hozzá másvalaki adataihoz.

## 6.4 Hibakezelés

Minden végpont try/catch blokkba van csomagolva. A várható hibaesetek egy konkrét http státuszkóddal és egy JSON hibaüzenettel térnek vissza:

| Státusz | Jelentés |
|---|---|
| 400 | Hiányzó kötelező mező (pl. nincs title vagy projectId) |
| 401 | Nincs érvényes munkamenet (nincs bejelentkezve) |
| 403 | A felhasználó be van jelentkezve, de nem tulajdonosa a kért erőforrásnak |
| 404 | Az erőforrás nem található |
| 409 | Ütközés (pl. már foglalt email címmel próbál regisztrálni) |
| 500 | Váratlan szerveroldali hiba |

Egy olyan váratlan hiba, amely elkerüli az adott végpont saját try/catch blokkját, egy záró Express hibakezelő middleware-ben kerül elkapásra, ez szerveroldalon naplózza a teljes hibát, a kliensnek viszont csak egy általános „Internal server error” üzenetet küld vissza, szándékosan elrejtve a stack trace-t és az adatbázis részleteket attól, aki a választ várja.

A közös végpont függvény hibaüzenetei a modul nevét tartalmazzák (pl. "Failed to fetch note modules", "Note title required"), így a kliens és a naplózás egyértelműen mutatja, melyik modul végpontja jelezte a hibát.

# 7. Frontend

## 7.1 Felépítés

A kliensoldal nem használ build folyamatot, csomagolót (bundler) vagy frontend keretrendszert: minden oldal egy sima HTML fájl, saját CSS fájllal és egy vagy több <script> címkével. Azok az oldalak, amelyek több modult is használnak, a scripteket egymás után, klasszikus módon töltik be, így azok közös globális hatókört osztanak meg. A workspace.html például először a workspace.js-t tölti be, amely a közös globális értékeket deklarálja:

API_BASE_URL: az API alapcíme

projectId: a megnyitott projekt azonosítója, amelyet az URL query stringjéből olvas ki (pl. workspace.html?id=5)

growCanvasIfNeeded(): a közös rajzvászon növelését végző segédfüggvény.

Ezt követi a kanbanModule.js, a flowchartModule.js, a pictureModule.js, és a noteModule.js, amelyek ugyanezeket az értékeket újradeklarálás nélkül használják. Ha a projekt azonosítója hiányzik az URL-ből, a workspace.js visszairányít a főoldalra, hiszen nincs mit megjeleníteni.

## 7.2 Fontosabb funkciók

**Drag and drop mozgatás és átméretezés:** a modulok és az alakzatok mozgatása és átméretezése Pointer Events segítségével történik (pointerdown / pointermove / pointerup). A Kanban kártyák oszlopok közötti áthelyezése

pedig a böngésző natív HTML5 Drag and Drop API-ját használja.

**Dinamikus DOM építés:** a táblák, kártyák, folyamatábrák, alakzatok és felugró ablakok (modal) futásidőben jönnek létre document.createElement és innerHTML használatával, nem statikus markupként léteznek az oldalon.

**Mentés HTML szerializálással:** egy tábla, jegyzet, kép vagy folyamatábra tartalma szerializált HTML-ként tárolódik az adatbázis data mezőjében. Mivel az eseménykezelők nem szerializálhatók, betöltéskor a kód újra felépíti őket: a Kanban modul a kártyákat a renderCard függvénnyel rendereli újra, a kép modul a loadPictureModules, a jegyzet modul a loadNoteModules, a folyamatábra modul pedig a rehydrateExistingContent függvénnyel köti vissza a mozgatást, a címke szerkesztést és a törlést, valamint építi újra az összekötések listáját az SVG vonalak data-from / data-to / data-h attribútumai hordozzák ugyanabban a data mezőben, így nem volt szükség új adatbázis oszlopra.

**Mentés eseményre:** a módosítások nem folyamatosan, hanem a művelet végén mentődnek: mozgatás vagy átméretezés végén, alakzat hozzáadásakor és törlésekor, összekötés létrehozásakor, címke / tartalom szerkesztésének befejezésekor (blur), illetve kártya felvételekor, törlésekor vagy áthelyezésekor.

**Szerkesztés helyben (inline edit):** a projektnevek és az alakzatcímkék egy dedikált kattintásra (ceruza ikon, dupla kattintás) szerkeszthetővé válnak (contentEditable), és a mező elhagyásakor vagy Enter megnyomásakor automatikusan mentődnek.

**Hitelesítési őr (auth guard):** a védett oldalak scriptje betöltéskor mehívja a GET /api/me végpontot, és érvénytelen munkamenet esetén a bejelentkezési oldalra irányít.

**Bővülő rajzvászon:** a közös munkaterület jobb és alsó szélén a vászon automatikusan nő, ahogy egy táblát vagy folyamatábrát a széle felé húznak, a túlcsordulást pedig a környező panel görgetősávja kezeli.

## 7.3 API használata

Minden modulfájl kis burkoló (wrapper) függvényeket definiál a fetch() köré az adott erőforráshoz, például a Kanban táblákhoz a fetchBoardsFromAPI, createBoardAPI és a deleteBoardAPI függvényeket, amelyeknek a folyamatábrás megfelelői a fetchChartsFromAPI, createChartAPI, saveChartAPI és a deleteChartAPI. A főoldal ugyanígy kezeli a projekteket. Ezek a burkolók egy helyre gyűjtik a kérések felépítését (metódus, fejlécek, JSON törzs), így a modul többi része egyetlen függvényhívással dolgozik, nem ismételgeti a fetch() beállításait. Hiba esetén a burkolók a konzolra naplózzák a problémát, és biztonságos értékkel (null vagy üres lista) térnek vissza, így egy sikertelen kérés nem állítja le az oldal többi részét.

Mivel az API és a kliens ugyanarról a szerverről (azonos eredetről) érkezik, a munkamenet cookie-t a böngésző minden kéréshez automatikusan csatolja, külön kliensoldali kód nélkül.

# 8. Telepítés és üzemeltetés

## 8.1 Szükséges környezet

Node.js 18 vagy újabb, az npm csomagkezelővel együtt (az Express 5 legalább a 18-as verziót igényli, a fejlesztés Node.js 22 alatt történt).

MySQL-kompatibilis adatbázis szerver (MySQL vagy MariaDB). A fejlesztés XAMPP csomag alatt történt, ahol elég a MySQL modult elindítani, az Apache csak akkor kell, ha az adatbázist phpMyAdminnal szeretnéd kezelni.

Modern asztali böngésző (pl. Chrome, Edge, Firefox).

## 8.2 Telepítés

A projektmappát (NotedApp) el kell helyezni a célgépen, majd a mappában ki kell adni a függőségek telepítéséhez szükséges parancsot. A node_modules mappát nem kell átmásolni, az npm install létrehozza:

cd NotedApp

npm install

Ez telepíti a package.json-ben felsorolt függőségeket: express, mysql2, bcrypt, express-session, cors, dotenv, valamint fejlesztői függőségként a nodemon-t.

(Ha nem engedi a rendszer a scriptek futtatását akkor az alábbit futtassa a terminálban, és ha kell akkor Y-t nyomjon, és próbálja meg újra a telepítést):

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

## 8.3 Konfiguráció

A projekt gyökérmappájában (a server.js és a package.json mellett) létre kell hozni egy .env fájlt az alábbi kulcsokkal:

| Változó | Kötelező | Alapértelmezett érték | Leírás |
|---|---|---|---|
| DB_HOST | nem | localhost | az adatbázis szerver címe |
| DB_USER | nem | root | adatbázis felhasználó |
| DB_PASSWORD | nem | (üres) | az adatbázis felhasználó jelszava |
| DB_NAME | nem | noted | Az adatbázis neve |
| SESSION_SECRET | igen | nincs | A munkamenet cookie aláírásához használt titkos kulcs, nélküle a szerver minden kérésre |

| | | | |
|---|---|---|---|
| | | | hibával válaszol |
| PORT | nem | 3000 | A szerver portja |

Példa .env fájl:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD =
DB_NAME=noted
SESSION_SECRET=<hosszú, véletlenszerű karakterlánc>
```

A SESSION_SECRET értékét érdemes legeneráltatni, nem kézzel kitalálni:

```
node -e
„console.log(require('crypto').randomBytes(32).toString('hex'))"
```

A .env fájl jelszót és titkos kulcsot tartalmaz, ezért soha nem szabad verziókezelőbe feltölteni vagy megosztani.

## 8.4 Adatbázis létrehozása

A 4.5 fejezetben szereplő SQL scriptet le kell futtatni az adatbázis szerveren (például a phpMyAdmin SQL fülén, vagy a mysql parancssori klienssel). A script létrehozza az adatbázist és mind a négy táblát (users, projects, modules, user_projects). A .env fájl DB_NAME értékének meg kell egyeznie a létrehozott adatbázis nevével.

## 8.5 Indítás

```
npm run dev        # nodemonnal indítja, mentéskor automatikusan újraindul (fejlesztéshez)
npm start          # sima indítás: node.server.js
```

Sikeres indításkor a konzolon megjelenik a „Noted server running on http://localhost:3000” üzenet. A működés gyors ellenőrzéséhez a böngészőben a http://localhost:3000/api/health címnek {„status”:”healthy”,”database”:”connected”} választ kell adnia, a http://localhost:3000 cím pedig a nyitóoldalt tölti be.

**Üzemeltetési tudnivalók és hibaelhárítás:**

**Munkamenetek újraindításkor elvesznek:** A szerver az express-session alapértelmezett memória alapú tárolóját használja, ezért a szerver újraindítása után minden felhasználónak újra be kell jelentkeznie. Fejlesztéshez ez elfogadható, éles használathoz tartós munkamenet tároló (pl. adatbázis alapú) kellene.

**A cookie nincs titkosított kapcsolatra korlátozva:** A kapcsolat HTTP-n megy, ezért éles környezetben HTTPS és a cookie secure beállítása szükséges.

**EADDRINUSE hiba:** induláskor azt jelenti, hogy a 3000-es portot már használja egy másik folyamat (gyakran egy korábban elindított, bezárt terminálban futó szerver).

Windows alatt a: netstat -ano | findstr:3000 megmutatja a folyamat azonosítóját, a: taskkill /F /PID <azonosító> pedig leállítja.

**Alapértelmezett XAMPP beállítás:** a root felhasználó jelszó nélkül, ami fejlesztéshez megfelel az üres DB_PASSWORD-nek, de éles környezetben jelszót kell beállítani.

# 9. Tesztelés

## 9.1 Tesztelési stratégia

A tesztelés két szinten történt:

**Kézi (felfedező) tesztelés:** minden funkció a megvalósítás után közvetlenül a böngészőben lett kipróbálva, a hibák diagnosztizálása a böngésző fejlesztői eszközeivel és a szerver konzolnaplójával történt. A felhasználói felület (drag and drop, alakzatok, összekötések) tesztelése kizárólag így történt.

**Automatizált API tesztek:** a Node.js beépített tesztfuttatójával írt tesztek, amelyek a szervert egy szabad porton elindítják, valódi HTTP-kéréseket küldenek a végpontoknak, majd ellenőrzik a státuszkódot és a válasz tartalmát.

## 9.2 Tesztesetek

Automatizált tesztek (test/ mappa, 7 teszt):

| Azonosító | Tesztelt terület | Ellenőrzött viselkedés |
|---|---|---|
| T1 | Health, statikus fájlok, CORS | GET /api/health 200 és healthy, a / HTML oldalt ad, az OPTIONS kérés 204-et és access-control-allow-origin: * fejlécet ad |

| | | |
|---|---|---|
| T2 | Regisztráció és profil | Hiányzó email 400, sikeres regisztráció 200 és munkamenet-cookie, a jelszó hashelve kerül tárolásra, létrejön az alapértelmezett projekt, foglalt email 409, hitelesítés nélkül /api/me 401, PUT /api/me üres névre 400, érvényes névre 200 (a név levágva), megszűnt felhasználóra 404 |
| T3 | Bejelentkezés, kijelentkezés | Hiányzó mező 400, ismeretlen email és rossz jelszó 401, helyes adatokkal 200, kijelentkezés után a korábbi cookie-val /api/me 401 |
| T4 | Projektek | Hitelesítés nélkül 401, üres név 400, létrehozás, a lista csak a saját projekteket adja, id szerint csökkenően, idegen felhasználó átnevezése és törlése 200 |
| T5 | Kanban táblák | Hitelesítés nélkül 401, hiányzó projectId/title 400, |

| | | |
|---|---|---|
| | | idegen felhasználó lekérdezése módosítása és törlése 403, a tulajdonos létrehozása listázása módosítása törlése sikeres, már törölt tábla újabb törlése 403 |
| T6 | Folyamatábrák | Mint T5, kiegészítve azzal, hogy idegen projektbe létrehozás is 403 |
| T7 | Kép és Jegyzet modulok | Mint T5 |
| T8 | Adatbázishibák | Lekérdezési hiba esetén 500 és a megfelelő hibaüzenet (/api/projects, /api/me), ping hiba esetén 500 és unhealthy, kapcsolódási hiba esetén 500 (health, login) |

Kézi tesztek:

| Azonosító | Leírás | Elvárt eredmény |
|---|---|---|
| M1 | Regisztráció után a users tábla megnézése phpMyAdminba | A password oszlopban bcrypt hash ($2$...) áll, |

| | | |
|---|---|---|
| | | létrejön egy alapértelmezett projekt és egy user_projects sor |
| M2 | Védett oldal (homepage, workspace) megnyitása kijelentkezve | Átirányítás a bejelentkezési oldalra |
| M3 | Projekt létrehozása, átnevezése (ceruza ikon), megnyitása | A név mentődik, a projekt a workspace.html?id=... oldalon nyílik meg |
| M4 | Kanban tábla létrehozása, kártya felvétele, kártya áthúzása másik oszlopba | A változás újratöltés után is megmarad |
| M5 | Folyamatábra létrehozása, alakzatok felvétele, címke szerkesztése, összekötés, mozgatás, átméretezés | A nyíl mozgatás közben követi az alakzatot, a méret és az összekötés újratöltés után is megmarad |
| M6 | Jegyzet modul létrehozása, cím és tartalom szerkesztése, modul törlése | A tartalom a "save" gombra kattintva elmentődik, a modul megjelenik, a cím elmentődik |
| M7 | Kép modul létrehozása, cím szerkesztése, kép beszúrása kép-URL segítségével | A modul megjelenik, a cím elmentődik, az "add" gombra kattintva a modulban a kép megjelenik |

| | | |
|---|---|---|
| M8 | Második fiókkal bejelentkezve a munkaterület megtekintése (a valódi adatbázison) | A második felhasználó üres projektlistát lát, az elsőé nem jelenik meg |
| M9 | Idegen projektazonosítóval hívott végpont a böngésző konzoljából: fetch('/api/boards?projectId=ID').then(r => r.status) | 403 |

## 9.3 Automatizált tesztek

Az automatizált tesztek a Node.js beépített tesztfuttatójával (node:test) és a node:assert/strict modullal készültek, külső tesztelési keretrendszer nélkül. A tesztek a valódi server.js Express alkalmazást importálják, és az app.locals.pool felülírásával egy memóriabeli adatbázis-helyettesítőt (mock pool) injektálnak a MySQL-kapcsolat helyére, így a futtatásukhoz nem szükséges adatbázis-szerver. A helyettesítő a lekérdezések szövege alapján ismeri fel az SQL utasításokat, és tömbökön hajtja végre a megfelelő műveletet, ismeretlen lekérdezésnél hibát dob, így egy elírt vagy hibásan meghívott lekérdezés a tesztet megbuktatja.

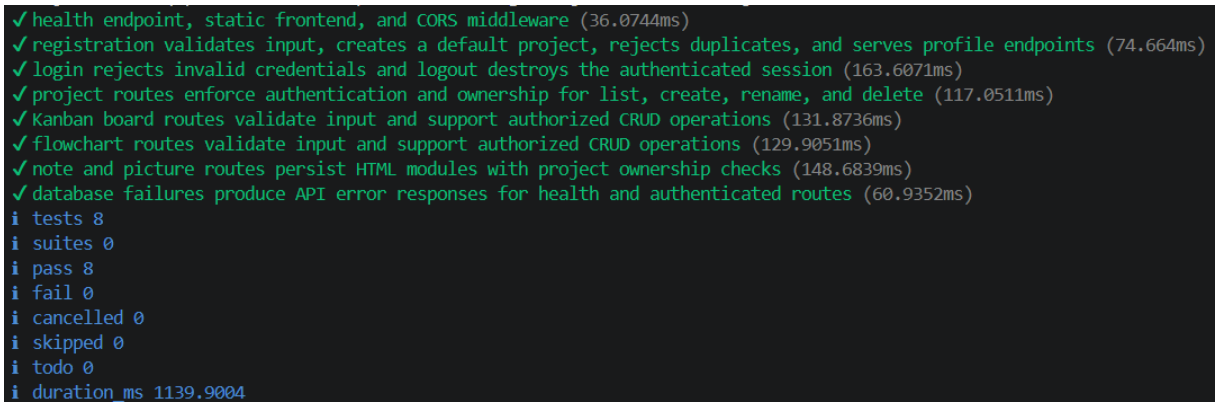
Minden teszt külön, szabad porton indítja el a szervert, valódi HTTP kéréseket küld neki, a munkamenet-cookie-t a kérések között tesztkód továbbítja, és ellenőrzi a státuszkódot és a válasz tartalmát. A tesztek lefedik a végpontokat, a hitelesítést és a munkamenet kezelést, a bemeneti validációt, a tulajdonjog ellenőrzést (idegen felhasználó adatai nem érhetők el), a hibakezelést és a CORS-t. Futtatás: npm test.

Korlátok: mivel at adatbázis helyettesítve van, az SQL lekérdezések tényleges végrehajtását, az adatbázissémát és az idegen kulcsokat az automatizált tesztek nem ellenőrzik, ezeekt kézi tesztek (M1, M8, M9) fedik valódi adatbázison. A kliensoldali kód (drag and drop, DOM építés) automatizált tesztje nem készült el.

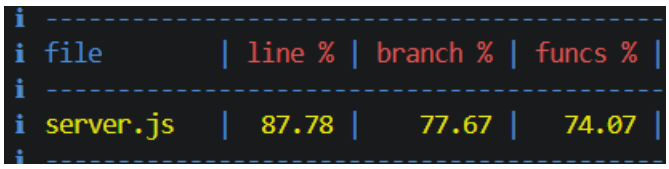
## 9.4 Teszteredmények

**Tesztkörnyezet:** Windows, Node.js 22.22.2, MySQL/MariaDB (XAMPP), böngésző: Chrome 155.0.8059.79 (hivatalos verzió, 64bit)

**Automatizált tesztek:**




Lefedettség: a node --test --experimental-test-coverage parancs sorlefedettségi jelentést ad.




**Kézi tesztek:** az M1-M9 esetek (elvárt eredményt adták).

## 9.5 Hibák és javításuk

Kanban táblák között kártyák mozgatása (a kártyáknak alapvetően egy táblához kell kötődniük, nem az összeshez).

**Javítás:** Kanban tábla id ellenőrzése kártya mozgatásnál.

Kanban Táblák és folyamatábrák összefolyása (egymás felülírása).

**Javítás:** Az adatbázisban megkülönböztetni a folyamatábrákat a kanban tábláktól (eddig nem voltak megkülönböztetve, a szerializált HTML-jük volt elmentve, ezért betöltésnél egymásba folytak és egymást felülírták).

# 10. Forráskód és projektstruktúra

## 10.1 Repository

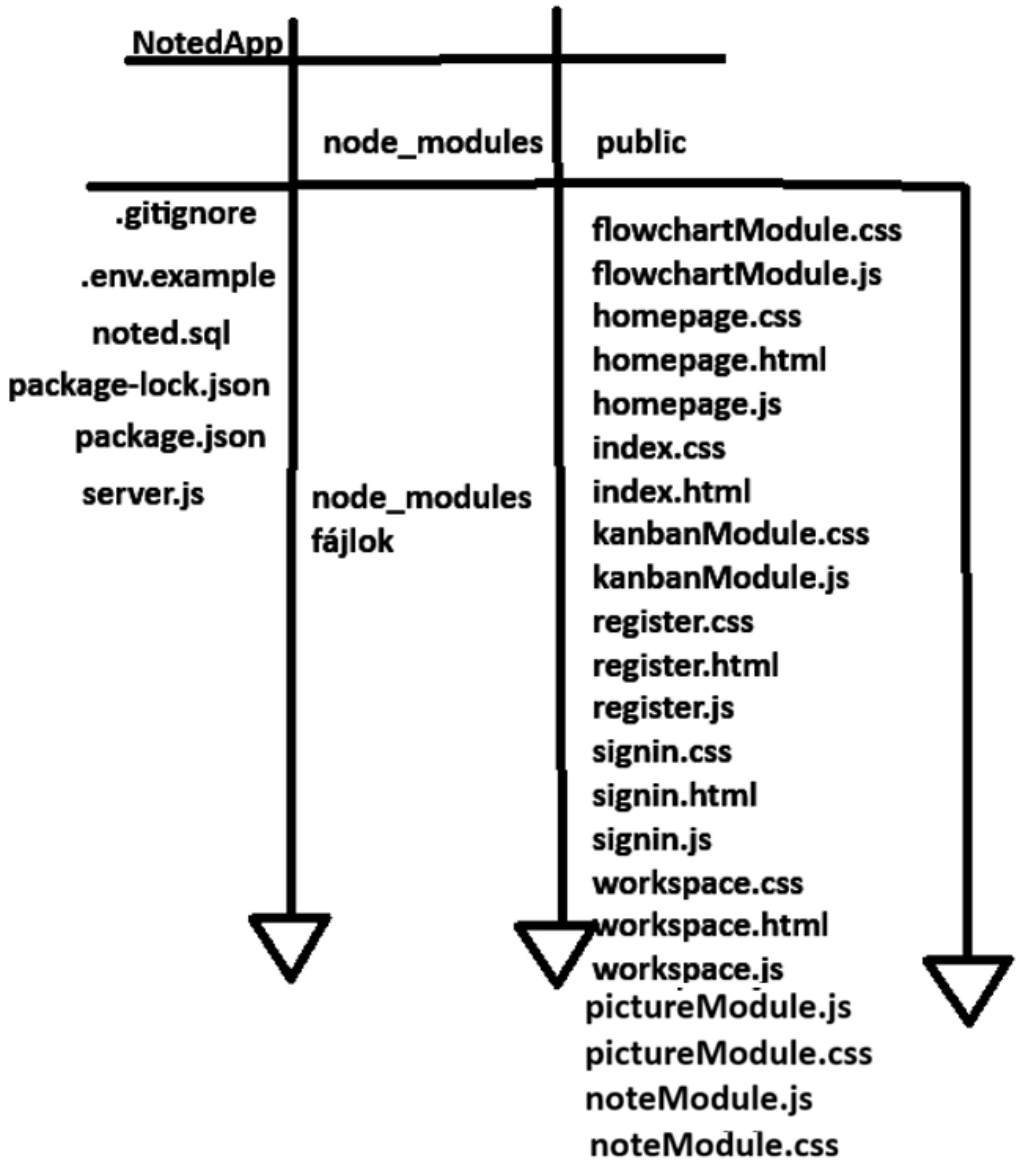
A projekt forráskódja Git verziókezelés alatt áll, a repository elérhetősége: https://github.com/PKornel01/Noted

A projekt klónozása és elindítása a 8. fejezetben leírt lépések szerint történik:

```
git clone https://github.com/PKornel01/Noted
cd NotedApp
npm install
```

A .env fájl szándékosan nem része a repositorynak, mert jelszót és titkos kulcsot tartalmaz. A szükséges kulcsok listáját a 8.3. fejezet és a repositoryban található .env.example fájl tartalmazza.

## 10.2 Könyvtárstruktúra




Valamint a public folder-ben vannak a használt képek is.

## 10.3 Fontosabb modulok

| Fájl | Szerepe | Fontosabb függvények / elemek |
|---|---|---|
| server.js | Az Express szerver: middleware-ek, hitelesítés, REST végpontok, adatbázis pool, hibakezelés | requireAuth, végpont csoportok (auth, projects, boards, charts, health, me) |
| workspace.js | A munkaterület közös beállításai, amelyeket a két modul újradeklarálás nélkül használ | API_BASE_URL, projectId, growCanvasIfNeeded, hitelesítési őr |
| kanbanModule.js | A Kanban modul teljes kliensoldali logikája | loadBoardsFromDatabase, persistBoardtoDB, renderCard, handleCardDrop, makeElementMovable, kártya és táblacím modal-ok |
| flowchartModule.js | A folyamatábra modul teljes kliensoldali logikája | loadChartsFromDatabase, persistChartToDB, makeInteractive, rehydrateExistingContent, createConnection, updateLine, edgePoint |
| pictureModule.js | A kép modul teljes | RequestPictureApi, createPictureRecord, savePictureRecord, |

| | | |
|---|---|---|
| | kliensoldali logikája | deletePictureRecord, buildPictureModule, persistPictureModule, attachPictureModuleEvents |
| NoteModule.js | A jegyzet modul teljes kliensoldali logikája | requestNoteApi, createNoteRecord, saveNoteRecord, deleteNoteRecord, buildNoteModule, persistNoteModule, attachNoteModuleEvents |
| homepage.js | A projektlista kezelése | initHomepage, loadProjects, renderProjects, createProjectBox, kijelentkezés |
| register.js / signing.js | Az űrlapok elküldése a regisztrációs és bejelentkezési végpontokra | űrlap submit eseménykezelők |

Minden modulfájl egyetlen fájlban tartalmazza az API hívásokat, a DOM építést és a mozgatási logikát. Ez a projekt méretéhez elegendő volt.

# 11. Csapatmunka

## 11.1 Csapattagok

Püspök Kornél

Orbán Regina

## 11.2 Feladatmegosztás

Püspök Kornél: Frontend és Backend fejlesztés, Dokumentáció.

Orbán Regina: UI / WEB Design, Frontend fejlesztés.

## 11.3 Használt projektmenedzsment eszközök

Github / Github desktop, a projekt további tervezésére pedig magát a projektet mikor a modulok készen lettek.

## 11.4 Saját feladat / hozzájárulás

Püspök Kornél: Weblapok scriptjeinek megírása, Flowchart és Kanban modul elkészítése / végpontjaik megírása, adatbázis létrehozása és bővítése, Backend scriptek megírása, Dokumentáció megírása, Automatizált tesztek írása, Manuális / kézi tesztelés.

Orbán Regina: UI design, modulok és weblapok finomítása, Weblap design, Jegyzet és Kép modulok elkészítése / végpontjaiknak megírása, program finomítása, Manuális / kézi tesztelés.

# 12. Összegzés

A NOTED! egy egyszerűen kezelhető projekttervező web alkalmazás, négy projekttervező modullal amik mentődnek a felhasználó saját fiókjához az adatbázisban.